

NAVEEN AUTOMATION LABS

DRIVEN BY CODE, DEFINED BY QUALITY



PLAYWRIGHT ADVANCED

Features & Best Practices

Date: _____

What We'll Cover

- **Auto-Waiting** - Smart waiting mechanism
- **Browser Contexts** - Isolation & parallelization
- **Network Interception** - API mocking & monitoring
- **Debugging Tools** - Trace viewer, inspector, video
- **Best Practices** - Industry-standard patterns

Auto-Waiting in Playwright

- Playwright **automatically waits** for elements before actions
- No need for **sleep()** or **explicit waits** in most cases!
- Checks multiple conditions: visible, stable, enabled, etc.

AUTO-WAIT CONDITIONS!

Before performing actions, Playwright automatically waits for:

- ✓ Element is *attached* to DOM
- ✓ Element is *visible*
- ✓ Element is *stable* (not animating)
- ✓ Element *receives events* (not covered)
- ✓ Element is *enabled* (for inputs/buttons)

Default timeout: 30 seconds

Auto-Wait Examples

{ } Auto-Wait in Action (TypeScript)

```
import { test, expect } from '@playwright/test';
test('auto-wait demo', async ({ page }) => {
  await page.goto('https://example.com');
  // ✓ Waits for element to be visible & clickable
  await page.click('button#submit');
  // ✓ Waits for element to be visible & attached
  await page.fill('input#email', 'test@test.com');
  // ✓ Waits for text to appear
  await expect(page.locator('.success'))
    .toHaveText('Success!');
  // ✓ Waits for element to be visible before getting text
  const text = await page.locator('.message').textContent();
});
// Custom timeout when needed
test('custom timeout', async ({ page }) => {
  // Wait up to 60 seconds for slow-loading element
  await page.click('button', { timeout: 60000 });
  // Set default timeout for this page
  page.setDefaultTimeout(60000);
});
```

Browser Contexts - Isolation

→ Browser Context = Isolated browser session

- Like **incognito mode** - own cookies, storage, cache
- Multiple contexts can run in **one browser** instance!
- Perfect for **parallel testing** & test isolation



Browser Context Examples

{ } Working with Browser Contexts

```
import { test, chromium } from '@playwright/test';

// Example 1: Multiple isolated sessions
test('multiple users', async () => {
  const browser = await chromium.launch();
  // User 1 session
  const context1 = await browser.newContext();
  const page1 = await context1.newPage();
  await page1.goto('https://example.com/login');
  // Login as user1...
  // User 2 session - completely isolated!
  const context2 = await browser.newContext();
  const page2 = await context2.newPage();
  await page2.goto('https://example.com/login');
  // Login as user2...
  // Both users can interact simultaneously!
  await context1.close();
  await context2.close();
  await browser.close();
});

// Example 2: Context with custom settings
test('custom context', async ({ browser }) => {
  const context = await browser.newContext({
    viewport: { width: 1280, height: 720 },
    locale: 'en-US',
    timezoneId: 'America/New_York',
    geolocation: { latitude: 40.7128, longitude: -74.0060 },
    permissions: ['geolocation'],
    userAgent: 'Custom User Agent'
  });
  const page = await context.newPage();
  await page.goto('https://example.com');
  await context.close();
});
```

```
});
```

REMEMBER:

Browser = Physical browser instance

Context = Isolated session (incognito-like)

Page = Individual tab/window

One Browser → Multiple Contexts → Multiple Pages

Always close contexts when done to free resources!

Network Interception

- **Intercept & modify** network requests/responses
- **Mock API responses** - test without backend!
- **Monitor network traffic** - debugging made easy
- **Block resources** - images, CSS, ads, etc.

Network Interception Examples

{ } API Mocking & Network Control

```
import { test } from '@playwright/test';  
// Example 1: Mock API Response  
test('mock API', async ({ page }) => {  
  // Intercept API call and return mock data  
  await page.route('**/api/users', async route => {  
    await route.fulfill({  
      status: 200,  
      contentType: 'application/json',  
      body: JSON.stringify([  
        { id: 1, name: 'John Doe', email: 'john@test.com' },  
        { id: 2, name: 'Jane Smith', email: 'jane@test.com' }  
      ])  
    });  
  });  
});
```

```

    ]);
  });
});
await page.goto('https://example.com/users');
// Page will receive mock data!
});
// Example 2: Block Images & CSS for faster tests
test('block resources', async ({ page }) => {
  await page.route('**/*', route => {
    const resourceType = route.request().resourceType();
    if (['image', 'stylesheet', 'font'].includes(resourceType)) {
      route.abort(); // Block these resources
    } else {
      route.continue(); // Allow others
    }
  });
  await page.goto('https://example.com');
});
// Example 3: Modify Request Headers
test('modify headers', async ({ page }) => {
  await page.route('**/api/**', async route => {
    const headers = route.request().headers();
    headers['Authorization'] = 'Bearer fake-token-123';
    await route.continue({ headers });
  });
  await page.goto('https://example.com');
});
// Example 4: Monitor Network Requests
test('monitor network', async ({ page }) => {
  // Listen to all requests
  page.on('request', request => {
    console.log('Request:', request.url());
  });
  // Listen to all responses
  page.on('response', response => {
    console.log('Response:', response.url(), response.status());
  });
  await page.goto('https://example.com');
});

```

Use network interception to:

- ✓ Test error scenarios (500, 404, timeout)
- ✓ Test without real backend (mock all APIs)
- ✓ Speed up tests (block images/ads)
- ✓ Test with different data scenarios
- ✓ Debug network issues easily



Debugging Tools

- **Trace Viewer** - Time-travel debugging!
- **Inspector** - Step through tests
- **Video Recording** - Visual test execution
- **Screenshots** - Capture failures automatically



Debugging Examples

{ } Debugging Tools Setup

```
import { test } from '@playwright/test';  
// 1. Trace Viewer - Records everything!  
test('with tracing', async ({ page, context }) => {  
  // Start tracing  
  await context.tracing.start({  
    screenshots: true,  
    snapshots: true,  
    sources: true  
  });  
  // Run your test  
  await page.goto('https://example.com');  
  await page.click('button');  
  // Stop and save trace  
  await context.tracing.stop({ path: 'trace.zip' });  
  // View: npx playwright show-trace trace.zip  
});  
// 2. Video Recording  
test('with video', async ({ browser }) => {
```

```

const context = await browser.newContext({
  recordVideo: {
    dir: './videos',
    size: { width: 1280, height: 720 }
  }
});

const page = await context.newPage();
await page.goto('https://example.com');
// Test actions...
await context.close();
// Video saved automatically in ./videos/
});

// 3. Screenshots on Failure (in playwright.config.ts)
export default {
  use: {
    screenshot: 'only-on-failure', // Auto screenshot on fail
    video: 'retain-on-failure', // Keep video on fail
    trace: 'retain-on-failure' // Keep trace on fail
  }
};

// 4. Manual Screenshot
test('manual screenshot', async ({ page }) => {
  await page.goto('https://example.com');
  // Full page screenshot
  await page.screenshot({ path: 'full-page.png', fullPage: true });
  // Element screenshot
  await page.locator('.header').screenshot({ path: 'header.png' });
});

```

! TRACE VIEWER IS AMAZING!

Playwright's Trace Viewer shows you:

- ✓ Every action taken during the test
- ✓ Screenshots at each step
- ✓ DOM snapshots before/after actions
- ✓ Network activity (all requests/responses)
- ✓ Console logs and errors

✓ Source code that was executed

Run: `npx playwright show-trace trace.zip`



Mobile + Device Emulation

{ } Device Emulation Examples

```
import { test, devices } from '@playwright/test';
// Test on iPhone 12
test('iPhone test', async ({ browser }) => {
  const iPhone = devices['iPhone 12'];
  const context = await browser.newContext({ ...iPhone });
  const page = await context.newPage();
  await page.goto('https://example.com');
  // Test runs with iPhone viewport & user agent!
  await context.close();
});
// Test on multiple devices
const deviceList = [
  'iPhone 12',
  'Pixel 5',
  'iPad Pro',
  'Desktop Chrome'
];
deviceList.forEach(deviceName => {
  test(`test on ${deviceName}`, async ({ browser }) => {
    const device = devices[deviceName];
    const context = await browser.newContext({ ...device });
    const page = await context.newPage();
    await page.goto('https://example.com');
    // Your test logic...
    await context.close();
  });
});
// Custom device configuration
test('custom device', async ({ browser }) => {
  const context = await browser.newContext({
    viewport: { width: 375, height: 667 },
    userAgent: 'Custom Mobile Browser',
```

```
hasTouch: true,  
isMobile: true  
});  
const page = await context.newPage();  
await page.goto('https://example.com');  
await context.close();  
});
```

Best Practices

✓ **BEST PRACTICE #1**

Use Strict Locators

Prefer specific, unique locators over generic ones:

✓ *Good:* `page.getByRole('button', { name: 'Submit' })`

✓ *Good:* `page.getByTestId('submit-btn')`

✗ *Bad:* `page.locator('button')` (too generic!)

✓ **BEST PRACTICE #2**

Use Built-in Locators

Playwright has semantic locators that are better than CSS:

- `getByRole()` - by ARIA role
- `getByText()` - by visible text
- `getByLabel()` - by label text
- `getByPlaceholder()` - by placeholder
- `getByTestId()` - by data-testid attribute

✓ **BEST PRACTICE #3**

Don't Use Sleep!

❌ *Never:* `await page.waitForTimeout(5000)`

✅ *Instead: Let Playwright auto-wait or use:*

- `await page.waitForLoadState('networkidle')`
- `await page.waitForSelector('.element')`
- `await expect(locator).toBeVisible()`

✅ **BEST PRACTICE #4**

Isolate Tests

Each test should be independent:

- Use `test.beforeEach()` for setup
- Use `test.afterEach()` for cleanup
- Don't depend on other tests' data
- Use browser contexts for isolation

✅ **BEST PRACTICE #5**

Use Page Object Model

Organize your code with POM pattern:

- Create page classes
- Encapsulate locators & actions
- Reuse code across tests
- Make tests readable & maintainable



Page Object Model Example

{ } POM Pattern in Playwright

```
// pages/LoginPage.ts
```

```
import { Page } from '@playwright/test';
```

```
export class LoginPage {
```

```
  readonly page: Page;
```

```
  // Locators
```

```
  readonly usernameInput = () => this.page.getByLabel('Username');
```

```
  readonly passwordInput = () => this.page.getByLabel('Password');
```

```
  readonly submitButton = () => this.page.getByRole('button', { name: 'Login' });
```

```
  readonly errorMessage = () => this.page.getByTestId('error-msg');
```

```
  constructor(page: Page) {
```

```
    this.page = page;
```

```
  }
```

```
  // Actions
```

```
  async goto() {
```

```
    await this.page.goto('/login');
```

```
  }
```

```
  async login(username: string, password: string) {
```

```
    await this.usernameInput().fill(username);
```

```
    await this.passwordInput().fill(password);
```

```
    await this.submitButton().click();
```

```
  }
```

```
  async getErrorMessage() {
```

```
    return await this.errorMessage().textContent();
```

```
  }
```

```
}
```

```
// tests/login.spec.ts - Using POM
```

```
import { test, expect } from '@playwright/test';
```

```
import { LoginPage } from '../pages/LoginPage';
```

```
test('successful login', async ({ page }) => {
```

```
  const loginPage = new LoginPage(page);
```

```
  await loginPage.goto();
```

```
  await loginPage.login('testuser', 'password123');
```

```
  await expect(page).toHaveURL('/dashboard');
```

```
});
```

```
test('invalid login', async ({ page }) => {
```

```
  const loginPage = new LoginPage(page);
```

```
  await loginPage.goto();
```

```
  await loginPage.login('baduser', 'badpass');
```

```
  const error = await loginPage.getErrorMessage();
```

```
  await expect(error).toContain('Invalid credentials');
```

```
});
```

X COMMON MISTAKES:

1. Using hard-coded waits:

→ *Never use `waitForTimeout()` unless absolutely necessary*

2. Not using test fixtures:

→ *Use `beforeEach/afterEach` for common setup*

3. Selecting elements poorly:

→ *Don't rely on brittle CSS selectors*

→ *Use semantic locators (`getByRole`, `getByText`)*

4. Not closing contexts/browsers:

→ *Always close to prevent resource leaks*

5. Ignoring parallel execution:

→ *Playwright runs tests in parallel by default*

→ *Ensure tests don't interfere with each other*